COLLEGE DEVOPS PRACTICAL EXAMINATION

Premium Calculator

Complete Project Summary, Technical Overview & Faculty Presentation Guide

Student Name:	Manas Chand
Roll Number:	23102C0030
Repository:	github.com/Manas-user/Premium-Calculator
Stack:	Java 17, Spring Boot, Maven, HTML/CSS/JS, Docker, Jenkins, Ansible

1. What is this Project? (In Simple Words)

Premium Calculator is a full-stack web application designed to calculate, manage, and track estimated insurance premiums. More importantly, it is built to demonstrate the **complete modern Software Development Lifecycle (SDLC) and DevOps pipeline** required in enterprise software engineering:

- **For the Customer/User:** A fast, responsive web portal where you enter a customer's name, choose a policy category (Life, Health, Vehicle, Travel), enter a coverage sum and duration, calculate estimated annual and total premiums, and download a text receipt voucher.
- **For Claims Officers:** A management dashboard to review submitted transactions, approve or reject them, and maintain an audit history.
- **For the DevOps Engineer / College Evaluation:** A complete CI/CD automation suite featuring version control (GitFlow), containerization (Docker), continuous integration (Jenkins and GitHub Actions), configuration management (Ansible), and operational health monitoring (Spring Boot Actuator).

> **Key Academic Disclaimer:** All calculation formulas and rates are demonstration estimates designed for educational grading and do not represent real-world commercial insurance underwriting quotes.

2. What the Application Does (Core Features)

1. Calculate Estimates

Calculates transparent annual and total premiums for 4 policy categories with strict mathematical precision using Java `BigDecimal`.

2. Transaction Lifecycle

Every calculation is saved with a unique ID (e.g. `TXN-A1B2C3D4`) in **SUBMITTED** state, and can be reviewed, approved, or rejected.

3. Instant Receipt Download

Generates an official formatted plain-text receipt voucher (`receipt-TXN-XXXXXXXX.txt`) ready for download.

4. System Health Heartbeat

Frontend continuously pings the backend Actuator endpoint (`/actuator/health`) to display a live green status badge (`Backend: UP`).

3. The Calculation Formula & Policy Rates

The backend implements a transparent, high-precision mathematical engine:

```
Annual Premium = Coverage Amount × Policy Rate
Estimated Total Premium = Annual Premium × Policy Term (in Years)
```

Policy Category	Illustrative Rate	Min / Max Coverage	Example (₹1,00,000 for 5 Years)
🌴 **Life Insurance**	1.20% (0.0120)	₹1,000 to ₹10,000,000	Annual: ₹1,200 | Total: ₹6,000
🤰 **Health Insurance**	2.40% (0.0240)	₹1,000 to ₹10,000,000	Annual: ₹2,400 | Total: ₹12,000
🚗 **Vehicle Insurance**	3.10% (0.0310)	₹1,000 to ₹10,000,000	Annual: ₹3,100 | Total: ₹15,500
✈️ **Travel Insurance**	1.60% (0.0160)	₹1,000 to ₹10,000,000	Annual: ₹1,600 | Total: ₹8,000

4. Technologies Used & Why We Used Them

Technology	Role	Why We Chose It
Java 17 LTS	Core Backend	Modern Long Term Support Java release with enhanced security, records, and performance.
Spring Boot 3.2.5	Web Framework	Enterprise standard for building production REST APIs, dependency injection, and embedded Tomcat.
Spring Boot Actuator	Monitoring	Provides production-ready health checks (`/actuator/health`) consumed by Docker, Jenkins, and Ansible.
HTML5, CSS3, Vanilla JS	Frontend UI	Zero framework bloat. Fast load times, responsive mobile-first design, direct REST API consumption.
JUnit 5 & Mockito	Automated Testing	33 comprehensive unit and integration tests verifying all calculations, API contracts, and edge cases.
Docker	Containerization	Multi-stage build (Temurin 17 JRE) with non-root security context (`appuser`) and health probes.
Jenkins CI/CD	Pipeline Automation	8-stage declarative pipeline (Checkout, Verify, Test, Package, Archive, Docker, Deploy, Health Check).
GitHub Actions	Cloud CI Runner	Runs automated tests and packages the JAR in the cloud on every push to `main` or `develop`.
Ansible	Configuration Mgmt	Agentless infrastructure automation playbook for idempotent deployment and health verification.

5. How We Built It (Step-by-Step Lifecycle)

1. **Step 1: Domain Modeling & Formula Engine:** Created Java enums (`PolicyType`, `TransactionStatus`), domain entities, and `PremiumCalculationService` using `BigDecimal` with `RoundingMode.HALF_UP` to prevent rounding errors.
2. **Step 2: REST API & Validation:** Implemented `PremiumCalculatorController` with Jakarta Bean Validation (`@Valid`, `@NotBlank`, `@DecimalMin`) and centralized advice (`GlobalExceptionHandler`) to return clean JSON error envelopes.
3. **Step 3: Persistence Abstraction:** Created a decoupled `TransactionRepository` interface with an in-memory thread-safe implementation (`ConcurrentHashMap`) ready for future database migration.
4. **Step 4: Responsive Frontend:** Built modern UI with live calculation summaries, synchronized term slider, quick amount buttons, receipt download, and transaction search/filter table.
5. **Step 5: Automated QA Suite:** Wrote 33 unit and integration tests. Executed test suite — **33 passed, 0 failed, 100% success rate**.
6. **Step 6: Version Control:** Initialized Git repository, followed GitFlow with `main` and `develop` branches, and pushed to GitHub: `https://github.com/Manas-user/Premium-Calculator`.

7. **Step 7: Multi-Stage Dockerfile:** Authored hardened two-stage Dockerfile that builds with JDK and runs on lightweight JRE as non-root user.
8. **Step 8: CI/CD & Configuration Management:** Authored 8-stage `Jenkinsfile`, GitHub Actions workflow (`.github/workflows/ci.yml`), and Ansible playbooks in `ansible/`.

6. Faculty Viva Voce & Presentation Guide

When presenting this project to your professor or external examiner, follow this structured 2-minute elevator pitch and be prepared for these typical questions:

> **Your 2-Minute Explanation Pitch to Faculty:**
> *"Good morning Sir/Madam. My project is 'Premium Calculator', developed by Manas Chand (Roll No. 23102C0030). It is a full-stack insurance estimation system designed to showcase the complete modern DevOps lifecycle. The backend is engineered with Java 17 and Spring Boot 3, providing high-precision financial calculation endpoints and a transaction state machine. The frontend is an interactive single-page application communicating via REST APIs. On the DevOps front, the application is covered by 33 automated unit and integration tests, containerized using a multi-stage Dockerfile running as a non-root user, automated through an 8-stage Jenkins pipeline and GitHub Actions, and orchestrated using Ansible with automated Actuator health verification."*

Expected Faculty Questions & Model Answers:

Q1: Why did you use BigDecimal instead of double or float for calculations?

Answer: *"In financial applications, `float` and `double` use binary floating-point representation, which causes imprecise decimal rounding (e.g., 0.1 + 0.2 != 0.3). `BigDecimal` guarantees exact arbitrary-precision arithmetic. We also enforce `RoundingMode.HALF_UP` scaled to 2 decimal places to meet currency standards."*

Q2: Why is a multi-stage Dockerfile better than a single-stage one?

Answer: *"A single-stage Dockerfile retains the entire JDK, Maven compiler, and source code in the final image, making it 800MB+ and vulnerable. In our multi-stage build, Stage 1 builds the JAR with JDK 17, but Stage 2 copies only the compiled JAR into a lightweight, hardened JRE 17 runtime, reducing image size to ~200MB and eliminating build tool attack surfaces."*

Q3: Why run the container as a non-root user?

Answer: *"By default, containers execute as `root`. If an attacker exploits an application vulnerability, they could potentially escape container boundaries with root privileges. In our Dockerfile, we explicitly create and switch to `appuser:appgroup` (UID 1001), adhering to the Principle of Least Privilege."*

Q4: What is the purpose of Spring Boot Actuator in your project?

Answer: *"Actuator exposes production-grade observability endpoints. We exposed `/actuator/health`, which returns a structured JSON heartbeat (`{"status":"UP"}`). This is actively polled by our frontend UI, Docker's `HEALTHCHECK` directive, the Jenkins deployment stage, and our Ansible playbook to guarantee zero-downtime health verification."*

Q5: What branching strategy did you follow in Git?

Answer: *"We adopted the GitFlow model using two primary branches: `main` (for production releases) and `develop` (for continuous integration and testing). We also followed the Conventional Commits convention (e.g. `feat:`, `chore:`, `ci:`)."*

7. Summary of Results

- **All 33 Tests Passing:** Surefire reports 33 tests executed with 0 failures, 0 errors, and 0 skipped.
- **Application Running:** Hosted at `http://localhost:8080` with active health check.
- **GitHub Repository:** Synchronized at `https://github.com/Manas-user/Premium-Calculator` with complete source history.
- **Cloud CI/CD:** GitHub Actions workflow actively compiling and verifying builds in the cloud.

Premium Calculator — College Practical DevOps Project © 2026 Manas Chand (Roll No. 23102C0030)